

LẬP TRÌNH WEB

Chương 2

Ngôn ngữ định kiểu CSS

GV: Bùi Văn Thương

Email: thuong.bui@ut.edu.vn

CÔNG CỤ HỌC TẬP

Code editor:
Microsoft Visual
Studio Code



Extension: Live
Server



NỘI DUNG CHÍNH

KIẾN THỨC CƠ BẢN VỀ CSS

PSEUDO – CLASS VÀ PSEUDO - ELEMENT

BIẾN TRONG CSS

FLEXBOX

GRID

XÂY DỰNG BỐ CỤC CHO WEBSITE

RESPONSIVE

CSS (Cascading Style Sheets) là ngôn ngữ định kiểu được dùng để tạo phong cách, bố cục, màu sắc và hiệu ứng cho các phần tử HTML trên trang web, giúp tách biệt nội dung và giao diện.

Các nội dung CSS cơ bản chúng ta sẽ tìm hiểu:

- Cú pháp của CSS
- Cách sử dụng CSS
- CSS Selectors

Cấu trúc tổng quát của một quy tắc CSS gồm 2 phần chính: Bộ chọn (Selector) và Khối khai báo (Declaration Block).

Cú pháp:

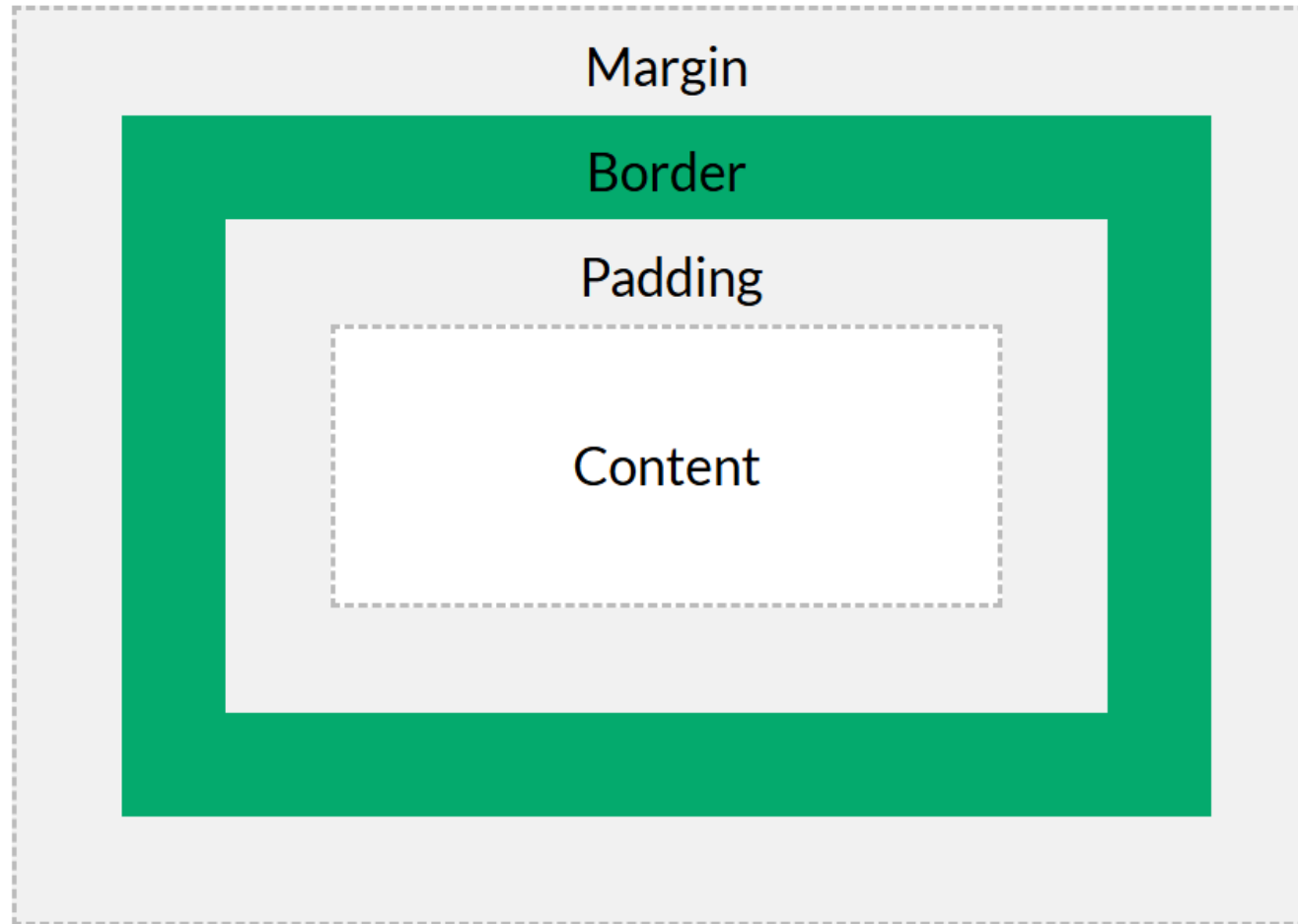
```
selector {  
    property1: value1;  
    property2: value2;  
    ...  
}
```

Ví dụ:

```
p {  
    color: blue; /* Thuộc tính color, giá trị blue */  
    font-size: 16px; /* Thuộc tính cỡ chữ, giá trị 16 pixel */  
    text-align: center; /* Thuộc tính căn lề, giá trị ở giữa */  
}
```

Ghi chú trong CSS được viết trong cặp ký hiệu `/*` và `*/`

Thuộc tính và giá trị định dạng cho khối:



Thuộc tính và giá trị định dạng cho khối:

margin: Khoảng cách vùng đệm xung quanh phần tử với các nội dung bên ngoài.

Dạng cơ bản gồm 4 thuộc tính:

Ví dụ:

```
section{  
    margin-top: 20px;  
    margin-bottom: 30px;  
    margin-left: 10px;  
    margin-right: 10px;  
}
```

Dạng tổng quát có thể có 1, 2, 3 hoặc 4 giá trị.

Ví dụ:

```
p {  
    margin: 10px 20px; /* trên-dưới 10px, trái-phải 20px */  
}
```

Lưu ý: Thuộc tính **margin: auto**; sẽ căn chỉnh phần tử vào giữa thành phần bao bọc bên ngoài theo chiều ngang.

Thuộc tính và giá trị định dạng cho khối:

padding: Khoảng cách vùng đệm giữa phần tử với các nội dung bên trong.

Ví dụ:

```
button {  
    padding: 10px 20px 30px; /* trên 10px, dưới 30px, trái-phải 20px */  
}  
section{  
    padding-top: 20px;  
    padding-bottom: 30px;  
    padding-left: 10px;  
    padding-right: 10px;  
}
```

Lưu ý: Thuộc tính padding mặc định sẽ làm cho kích thước của phần tử hiện hành tăng lên. Để không làm ảnh hưởng đến kích thước của phần tử này ta có thể thêm thuộc tính box-sizing với giá trị border-box:

```
box-sizing: border-box;
```

Thuộc tính và giá trị định dạng cho khối:

border: Dùng để định dạng đường viền cho thành phần.

Cú pháp chung:

```
tag {  
    border-vị trí: [màu sắc] [kiểu] [độ rộng];  
}
```

Các thuộc tính định dạng cho border:

- border-color: màu cho đường viền.
- border-style: kiểu cho đường viền.
- border-width: độ rộng cho đường viền.
- border-radius: bán kính đường bo góc.

Ví dụ:

```
table {  
    border: rgb(9, 9, 206) solid 1px;  
    border-radius: 6px;  
}
```

Lưu ý: Giống thuộc tính padding, border cũng làm tăng kích thước của phần tử. Ta cũng có thể xử lý trường hợp này bằng thuộc tính box-sizing với giá trị border-box.

Một số thuộc tính khác:

Thuộc tính	Giá trị	Ý nghĩa
color	Mã màu/tên màu/màu rgb	Định dạng màu sắc cho thành phần.
font-size	Giá trị (px/%/pt/em)	Cỡ chữ
font-family	Kiểu chữ	Định dạng font chữ cho thành phần
font-style	italic, oblique, normal	Định dạng kiểu chữ cho thành phần
letter-spacing	Giá trị (px/%/pt/em)	Khoảng cách các ký tự cho thành phần
line-height	Giá trị số có hoặc không có đơn vị	Khoảng cách các dòng cho thành phần
text-decoration	blink, line-through, overline, underline, none	Xác định thành phần có gạch dưới hay không

Một số thuộc tính khác:

Thuộc tính	Giá trị	Ý nghĩa
word-spacing	Giá trị (px/%/pt/em)	Khoảng cách giữa các từ
background	Mã màu hoặc url() chứa ảnh	Hình nền cho thành phần
height	Giá trị (px/%/pt/em)	Chiều cao thành phần
width	Giá trị (px/%/pt/em)	Chiều rộng thành phần
text-align	left/right/center/justify	Căn chỉnh nội dung (mặc định là left)
display	block/inline/none	Xác định hiển thị cho thành phần
position	relative/absolute/fixed/static	Xác định vị trí tương đối và tuyệt đối cho thành phần, vị trí này phụ thuộc vào các giá trị khai báo của thành phần và thành phần bao ngoài nó.

CÁCH SỬ DỤNG CSS

Cách 1: Inline CSS

Viết mã CSS trực tiếp vào trong thẻ HTML thông qua thuộc tính style.

```
<body>  
  <h2 style="color: red; font-size: 20px;">  
    CSS Nội dòng  
  </h2>  
</body>
```



CSS Nội dòng

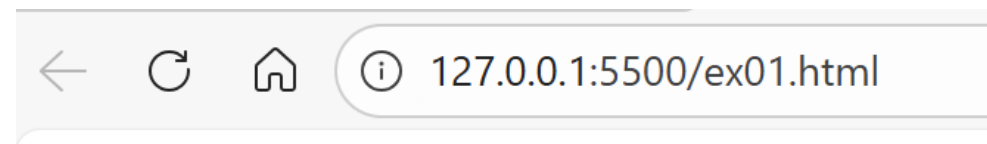
Ưu điểm: Nhanh, tiện khi muốn sửa nhanh một phần tử duy nhất.

Nhược điểm: Làm code HTML cực kỳ rối và khó bảo trì.

Cách 2: Internal CSS

Viết toàn bộ mã CSS nằm gọn trong cặp thẻ `<style>` và đặt nó bên trong thẻ `<head>` của trang HTML.

```
<head>
  <meta charset="UTF-8">
  <style>
    h2 {
      color:blueviolet;
      font-style: italic;
    }
  </style>
</head>
<body>
  <h2>CSS Nội trang</h2>
</body>
```



CSS Nội trang

Ưu điểm: Giúp quản lý toàn bộ style của trang HTML tại 1 nơi.

Nhược điểm: Khó quản lý đồng bộ khi có quá nhiều trang HTML.

Cách 3: External CSS

Ta viết CSS ra một file riêng (đuôi .css) và dùng thẻ <link> để gọi nó vào HTML.

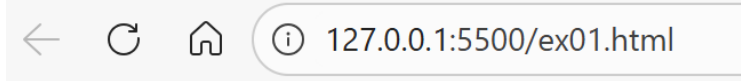
Nội dung file HTML

```
<head>
  <meta charset="UTF-8">
  <title>Trang web đầu tiên của tôi</title>
  <link rel="stylesheet" type="text/css" href="style.css">
</head>
<body>
  <h2>CSS bên ngoài trang</h2>
</body>
```

Nội dung file style.css

```
h2 {
  color: chocolate;
  text-transform: uppercase;
}
```

Kết quả



CSS BÊN NGOÀI TRANG

Đây là cách viết CSS chuyên nghiệp giúp tái sử dụng cho nhiều trang HTML, code gọn gàng, dễ xử lý. Đồng thời trình duyệt sẽ lưu file CSS này vào bộ nhớ đệm (cache), giúp web tải nhanh hơn ở các lần truy cập sau

Chọn theo tên thẻ:

Tác động lên tất cả các thẻ cùng loại.

```
<!DOCTYPE html>
<html lang="vi">
<head>
  <meta charset="UTF-8">
  <title>HTML và CSS</title>
  <link
    rel="stylesheet"
    type="text/css"
    href="style.css"
  >
</head>
<body>
  <h2>Lập trình Web</h2>
  <h3>Lập trình Front-end</h3>
  <h3>Lập trình Back-end</h3>
</body>
</html>
```

Nội dung file style.css

```
h2 {
  color:rgb(81, 38, 7);
  text-transform: uppercase;
}
h3{
  color:darkgreen;
}
```

Kết quả

LẬP TRÌNH WEB

Lập trình Front-end

Lập trình Back-end

Chọn thẻ lồng nhau:

Dùng cho các thẻ nằm trong thẻ khác trong cấu trúc HTML.

Mã HTML

```
<body>
  <h2>Lập trình web</h2>
  <nav>
    <h2>Lập trình PHP</h2>
    <h3>Lập trình JavaScript</h3>
  </nav>
</body>
```

Mã CSS

```
nav h2{
  color:rgb(8, 85, 85);
}
```

Kết quả

Lập trình web

Lập trình PHP

Lập trình JavaScript

Chọn theo class:

Dùng cho một nhóm phần tử. Bắt đầu bằng dấu chấm.

Mã HTML

```
<body>
  <h2 class="title">Lập trình Front-end</h2>
  <ol class="element">
    <li>HTML</li>
    <li>CSS</li>
    <li class="special">JavaScript</li>
  </ol>
</body>
```

LẬP TRÌNH FRONT-END

1. HTML
2. CSS
- 3. JavaScript**

Mã CSS

```
.title {
  color:rgb(81, 7, 43);
  text-transform: uppercase;
}
.element{
  color:rgb(33, 4, 252);
}
.special{
  font-weight: bold;
  color:crimson;
}
```

Chọn theo id:

Dùng cho duy nhất 1 phần tử. Bắt đầu bằng **dấu thăng #**

Mã HTML

```
<body>
  <form>
    <p>Bạn đồng ý xóa nội dung này?</p>
    <button class="btn">Đồng ý</button>
    <button class="btn" id="btn-cancel">
      Bỏ qua
    </button>
  </form>
</body>
```

Mã CSS

```
.btn{
  background:rgb(2, 98, 49);
  padding:5px 10px;
  border:none;
  color:white;
}
#btn-cancel{
  background:rgb(155, 161, 161);
}
```

Bạn đồng ý xóa nội dung này?

Đồng ý

Bỏ qua

Nhóm selectors:

Các selector cách nhau bởi dấu phẩy.

Mã HTML

```
<body>
  <h2>Lập trình PHP</h2>
  <h3>Lập trình Java</h3>
  <p>Lập trình Python</p>
</body>
```

Mã CSS

```
h2, h3, p {
  color:blue;
}
```

← ↻ 🏠 ⓘ 127.0.0.1:5500/ex01.html

Lập trình PHP

Lập trình Java

Lập trình Python

Universal selector *:

Áp dụng chung cho trang HTML

Mã HTML

```
<body>
  <h2>Lập trình PHP</h2>
  <h3>Lập trình Java</h3>
  <p>Lập trình Python</p>
</body>
```

Mã CSS

```
* {
  color:rgb(246, 139, 0);
  font-style: italic;
}
```

Lập trình PHP

Lập trình Java

Lập trình Python

Pseudo-class (Lớp giả) là những từ khóa được thêm vào sau bộ chọn (selector) để xác định một trạng thái đặc biệt của phần tử đó.

Cú pháp của Pseudo-class:

```
selector: pseudo-class {  
    property1: value1;  
    property2: value2;  
    ...  
}
```

Các nhóm Pseudo-class phổ biến nhất:

- Nhóm Tương tác
- Nhóm Cấu trúc
- Nhóm Trạng thái Form

Nhóm Pseudo-class tương tác:

Đây là nhóm giúp trang web trông "sinh động" hơn khi người dùng di chuột hoặc click vào.

- `:hover`: Khi người dùng di chuột qua phần tử (thường dùng cho nút bấm hoặc link).
- `:active`: Khi phần tử đang được nhấn giữ chuột.
- `:focus`: Khi phần tử đang được chọn (chẳng hạn như đang gõ chữ vào ô Input).

Ví dụ:

```
button:hover {  
    background-color: yellow; /* Di chuột vào là đổi sang màu vàng */  
}
```

Nhóm Pseudo-class cấu trúc:

Nhóm này giúp chọn các phần tử dựa trên vị trí của chúng trong "phả hệ" DOM mà không cần đặt thêm Class hay ID.

- :first-child: Chọn phần tử con đầu tiên.
- :last-child: Chọn phần tử con cuối cùng.
- :nth-child(n): Chọn phần tử thứ n.

Ví dụ:

```
/* Tô màu vàng cho các dòng chẵn trong bảng */  
tr:nth-child(even) {  
    background-color: #eddf72;  
}
```

1	2	3
1	2	3
1	2	3
1	2	3

Nhóm Pseudo-class trạng thái form:

Dùng để kiểm tra trạng thái của các ô nhập liệu trong Form.

- `:checked`: Khi một checkbox hoặc radio button được chọn.
- `:disabled`: Khi một ô nhập liệu bị khóa không cho dùng.
- `:invalid`: Khi dữ liệu người dùng nhập vào bị sai định dạng (chẳng hạn gõ email thiếu chữ @).

Ví dụ:

```
<body>
  <input type="checkbox" id="task1">
  <label for="task1">Hoàn thành bài tập CSS</label>
</body>
```

```
input:checked + label {
  color: grey;
  text-decoration: line-through;
  font-style: italic;
}
```

Kết quả:

☐ Hoàn thành bài tập CSS



☒ ~~Hoàn thành bài tập CSS~~

Pseudo - element (phần tử giả) thường dùng hai dấu :: là để tạo ra hoặc tác động vào một phần cụ thể của phần tử đó mà không cần thêm code HTML.

Cú pháp chuẩn:

```
selector::pseudo-element {  
    property1: value1;  
    property2: value2;  
    ....  
}
```

Các nhóm Pseudo - element phổ biến nhất:

- Nhóm Văn bản
- Nhóm tương tác người dùng

Nhóm Pseudo - element Văn bản:

Nhóm này giúp tạo ra các phong cách trình bày chuyên nghiệp giống như trên các mặt báo giấy truyền thống.

- `::first-letter`: Tác động vào chữ cái đầu tiên. Thường dùng để làm "Drop Cap" (chữ cái đầu dòng to đùng).
- `::first-line`: Tác động vào toàn bộ dòng đầu tiên của đoạn văn.

```
p::first-letter {  
  font-size: 320%;  
  float: left;  
  padding-right: 2px;  
  margin-top: -8px;  
  color: #ff5733;  
}
```

Địa lan tràn mộng Sa Pa được chào bán với giá hàng trăm triệu đồng một chậu vẫn "cháy hàng" do nguồn cung giảm, nhu cầu tăng cao trước Tết.

Nhóm Pseudo - element tương tác người dùng:

Đây là những chi tiết nhỏ nhưng giúp website trông cực kỳ cao cấp.

- `::selection`: Thay đổi màu nền và màu chữ khi người dùng bôi đen (quét chuột) văn bản.
- `::placeholder`: Thay đổi kiểu dáng cho dòng chữ gợi ý trong ô nhập liệu (input).

```
/* Đổi màu khi bôi đen chữ thành nền đen chữ vàng */  
::selection {  
    background: black;  
    color: yellow;  
}  
/* Đổi màu chữ gợi ý trong ô nhập liệu */  
input::placeholder {  
    color: rgb(187, 2, 215);  
    font-style: italic;  
    opacity: 0.5;  
}
```

Nhập tên của bạn

Bây **bôi chọn vào nội** dung này

Khai báo biến:

Biến thường được khai báo trong bộ chọn :root (đây là bộ chọn cao nhất, tương ứng với thẻ <html>), giúp biến có tác dụng trên toàn bộ trang web.

- Tên biến: Bắt buộc bắt đầu bằng hai dấu gạch ngang --.
- Cú pháp: --tên-biến: giá-trị;

Sử dụng biến:

- Để gọi biến ra dùng, ta sử dụng hàm var().
- Cú pháp: property: var(--tên-biến);

Ví dụ:

```
<body>
  <h3>Lập trình web</h3>
  <p>Biến trong CSS</p>
</body>
```

← ↻ 🏠 ⓘ 127.0.0.1:5500/ex01.html

Lập trình web

Biến trong CSS

```
:root {
  --primary-color: rgb(255, 123, 0);
  --secondary-color: rgb(0, 56, 78);
}
h3{
  color:var(--primary-color);
}
p{
  color:var(--secondary-color);
}
```

NHẬN XÉT:

Bằng việc sử dụng biến trong CSS để quản lý những giá trị giống nhau lặp lại, ta có thể dễ dàng thay đổi các cài đặt CSS của cả trang web một cách nhanh chóng giúp cho code của chúng ta trở nên thông minh, sạch sẽ và dễ bảo trì hơn.

FLEXBOX

Flexbox dùng để sắp xếp các phần tử theo hàng ngang hoặc hàng dọc. Nó cực kỳ mạnh mẽ trong việc căn giữa và phân bổ khoảng cách.

Các thuộc tính cốt lõi:

- **display: flex;** Kích hoạt chế độ Flex.
- **justify-content:** Căn chỉnh dọc theo trục chính (trái, phải, giữa, giãn đều).
- **align-items:** Căn chỉnh theo trục vuông góc.
- **flex-direction:** Xếp các phần tử theo hàng (row – mặc định) hay cột (column).
- **flex-wrap:** Cho phép phần tử tràn xuống dòng hay không.
- **flex-flow:** là sự kết hợp của flex-direction và flex-wrap.

Mã HTML

```
<nav>
  <ul>
    <li><a href="#">Trang chủ</a></li>
    <li><a href="#">Giới thiệu</a></li>
    <li><a href="#">Dịch vụ</a></li>
    <li><a href="#">Liên hệ</a></li>
  </ul>
</nav>
```

Kết quả hiển thị

Trang chủ Giới thiệu Dịch vụ Liên hệ

Mã CSS

```
ul{
  list-style-type: none;
  margin: 5px 0;
  padding: 0;
  background-color: antiquewhite;
  display: flex;
  justify-content: center;
}
li{
  margin: 0 5px;
  padding: 8px;
  font-size: 18px;
}
a{
  text-decoration: none;
  color: #7c1818;
}
```

Thuộc tính **justify-content** có các giá trị:

- **flex-start** (mặc định): Các phần tử dồn về phía bắt đầu của trục chính.
- **flex-end**: Các phần tử dồn về phía kết thúc của trục chính (thường là bên phải).
- **center**: Các phần tử được gom lại và đặt chính giữa hàng. Đây là cách nhanh nhất để căn giữa menu hoặc các nút bấm.
- **space-around**: Khoảng trống được chia đều vào hai bên của mỗi phần tử.
- **space-between**: Phần tử đầu tiên dính sát lề trái, phần tử cuối cùng dính sát lề phải. Toàn bộ khoảng trống dư thừa được chia đều vào giữa các phần tử còn lại.
- **space-evenly**: Khoảng trống được chia sao cho tất cả các khoảng cách đều bằng nhau tuyệt đối (khoảng cách giữa các phần tử = khoảng cách đến lề).

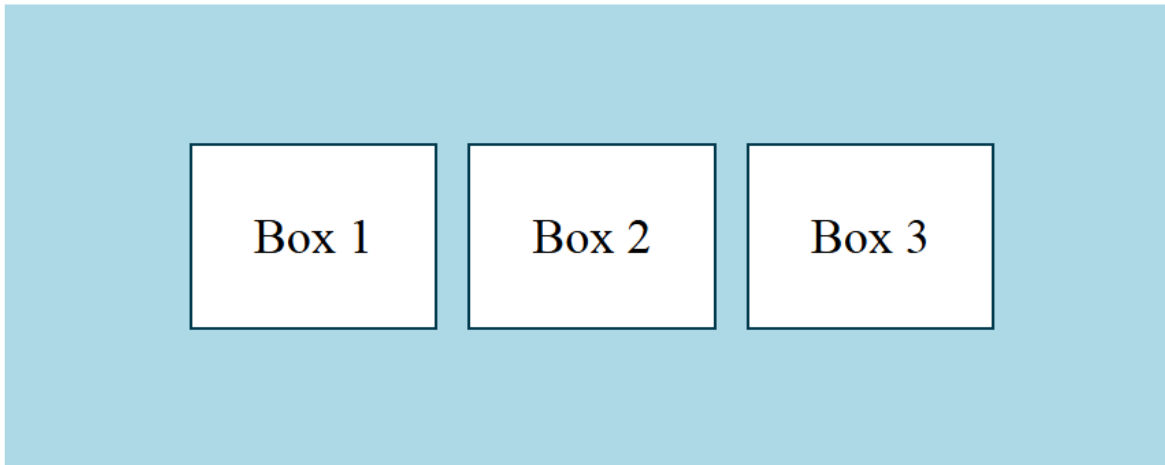
Thuộc tính **align-item** có các giá trị:

- **stretch**: Các phần tử con sẽ tự động "giãn" ra để lấp đầy toàn bộ chiều cao (hoặc chiều rộng, tùy trục) của phần tử cha.
- **center**: Các phần tử được căn chính giữa theo trục vuông góc. Kết hợp với *justify-content: center* để căn một phần tử vào đúng tâm của khung hình.
- **flex-start**: Các phần tử được dồn về phía bắt đầu của trục vuông góc. Nếu dàn hàng ngang, chúng sẽ "dính" lên đỉnh.
- **flex-end**: Các phần tử được dồn về phía kết thúc của trục vuông góc. Nếu dàn hàng ngang, chúng sẽ "chạm sàn".
- **baseline**: Các phần tử được căn chỉnh sao cho đường chân chữ (baseline) của chúng thẳng hàng với nhau.

Mã HTML

```
<section>
  <div>Box 1</div>
  <div>Box 2</div>
  <div>Box 3</div>
</section>
```

Kết quả hiển thị



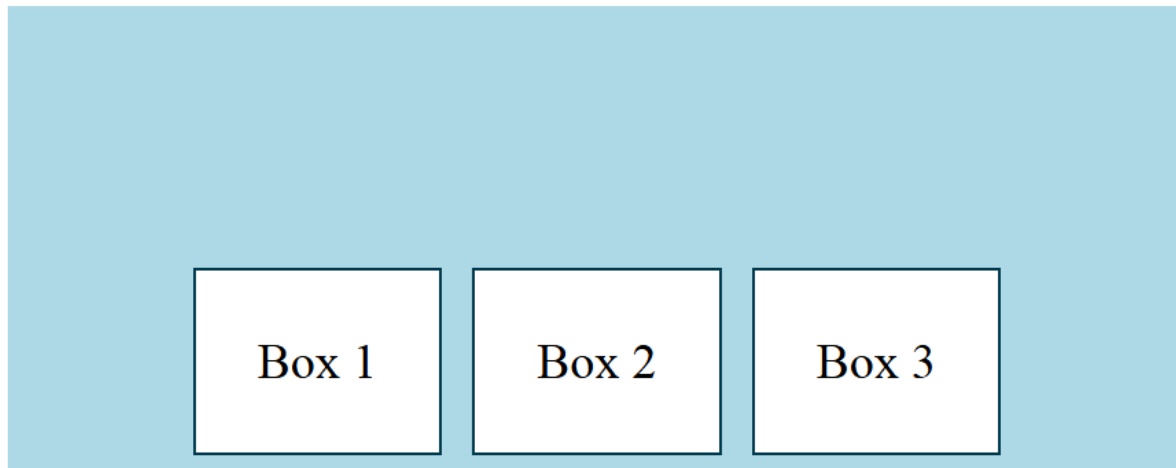
Mã CSS

```
section{
  display: flex;
  justify-content: center;
  align-items: center;
  background-color: lightblue;
  height: 150px;
}
section div{
  background-color: white;
  padding: 20px;
  margin: 5px;
  border: 1px solid rgb(2, 59, 78);
}
```

Mã HTML

```
<section>
  <div>Box 1</div>
  <div>Box 2</div>
  <div>Box 3</div>
</section>
```

Kết quả hiển thị



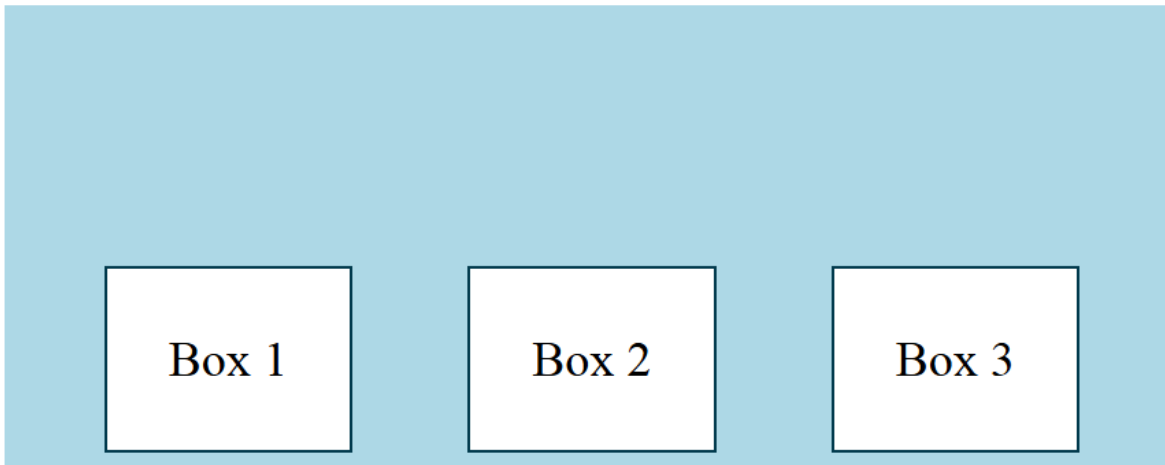
Mã CSS

```
section{
  display: flex;
  justify-content: center;
  align-items: end;
  background-color: lightblue;
  height: 150px;
}
section div{
  background-color: white;
  padding: 20px;
  margin: 5px;
  border: 1px solid rgb(2, 59, 78);
}
```

Mã HTML

```
<section>
  <div>Box 1</div>
  <div>Box 2</div>
  <div>Box 3</div>
</section>
```

Kết quả hiển thị



Mã CSS

```
section{
  display: flex;
  justify-content: space-between;
  align-items: end;
  background-color: lightblue;
  height: 150px;
}
section div{
  background-color: white;
  padding: 20px;
  margin: 5px;
  border: 1px solid rgb(2, 59, 78);
}
```

Chú ý:

- **align-items**: Dùng cho **một hàng** duy nhất. Nó căn chỉnh các phần tử bên trong hàng đó.
- **align-content**: Chỉ dùng khi có **nhiều hàng** (khi dùng flex-wrap: wrap). Nó căn chỉnh khoảng cách giữa các hàng với nhau.

Căn chỉnh một nội dung vào giữa màn hình:

Mã CSS

```
.cha {  
  display: flex;  
  justify-content: center; /* Giữa theo chiều ngang */  
  align-items: center;    /* Giữa theo chiều dọc */  
  height: 100vh;         /* Chiều cao bằng toàn bộ màn hình */  
}
```

Thuộc tính **flex-direction** có các giá trị:

- **row** (Mặc định): Xếp từ trái sang phải.
- **row-reverse**: Xếp từ phải sang trái (phần tử số 1 sẽ nằm ngoài cùng bên phải).
- **column**: Xếp từ trên xuống dưới (biến hàng ngang thành cột dọc).
- **column-reverse**: Xếp từ dưới lên trên.

Mã HTML

```
<section>
  <div>Box 1</div>
  <div>Box 2</div>
  <div>Box 3</div>
</section>
```

Kết quả hiển thị



Mã CSS

```
section{
  display: flex;
  justify-content: center;
  flex-direction: column-reverse;
  background-color: lightblue;
}
section div{
  background-color: white;
  padding: 10px;
  margin: 5px;
  border: 1px solid rgb(2, 59, 78);
  width: 80px;
}
```

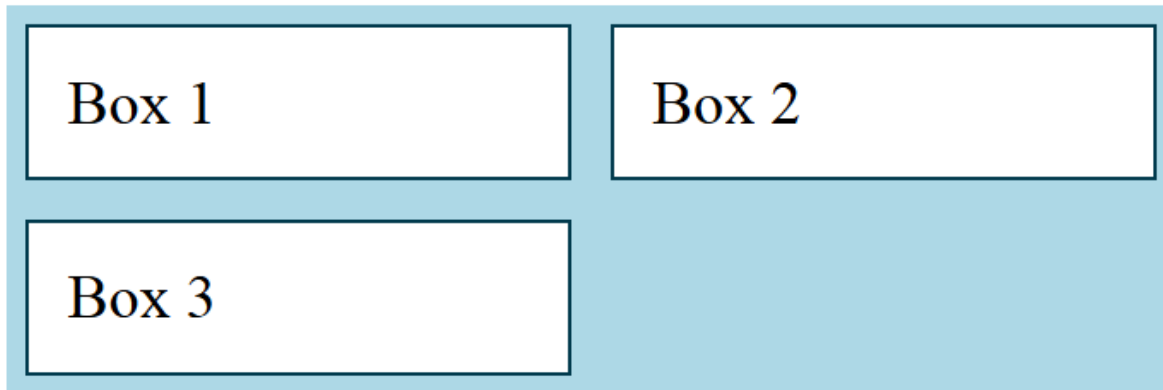
Thuộc tính **flex-wrap** có các giá trị:

- **nowrap** (Mặc định): Tất cả dồn vào 1 hàng. Có thể gây tràn hoặc méo phần tử.
- **wrap**: Nếu hết chỗ, phần tử tự động nhảy xuống hàng tiếp theo. Đây là giá trị "vàng" để làm danh sách sản phẩm trên Web.
- **wrap-reverse**: Thay vì xuống dưới, phần tử mới sẽ nhảy lên... phía trên.

Mã HTML

```
<section>
  <div>Box 1</div>
  <div>Box 2</div>
  <div>Box 3</div>
</section>
```

Kết quả hiển thị



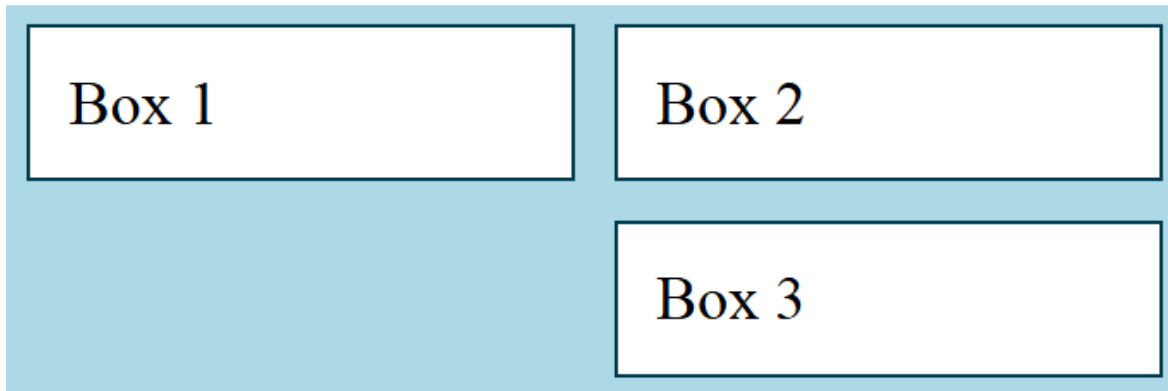
Mã CSS

```
section{
  display: flex;
  justify-content: flex-start;
  flex-wrap: wrap;
  background-color: lightblue;
}
section div{
  background-color: white;
  padding: 10px;
  margin: 5px;
  border: 1px solid rgb(2, 59, 78);
  width: 120px;
}
```

Mã HTML

```
<section>
  <div>Box 1</div>
  <div>Box 2</div>
  <div>Box 3</div>
</section>
```

Kết quả hiển thị



Mã CSS

```
section{
  display: flex;
  justify-content: flex-end;
  flex-wrap: wrap;
  background-color: lightblue;
}
section div{
  background-color: white;
  padding: 10px;
  margin: 5px;
  border: 1px solid rgb(2, 59, 78);
  width: 120px;
}
```

Thuộc tính **flex-flow**:

flex-flow là sự kết hợp của flex-direction và flex-wrap.

Cú pháp: flex-flow: [direction] [wrap];

Ví dụ:

```
/* Thay vì viết: */  
.container {  
  flex-direction: row;  
  flex-wrap: wrap;  
}  
/* Ta chỉ cần viết: */  
.container {  
  flex-flow: row wrap;  
}
```

Grid cho phép làm việc cả với hàng và cột cùng lúc, tư duy của grid là kẻ các đường lưới như bàn cờ và đặt các phần tử vào các ô.

Khi nào dùng Grid, khi nào dùng Flexbox?

- Dùng **Grid** cho bố cục tổng thể trang web (Header, Sidebar, Content...).
- Dùng **Flexbox** cho các thành phần bên trong (Căn giữa nội dung trong một ô, sắp xếp các icon trong thanh menu...).

Thuật ngữ cốt lõi

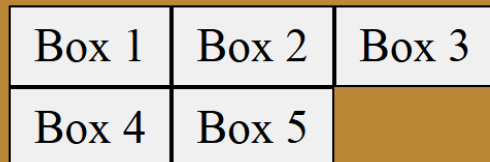
- Grid Container: Phần tử cha (khai báo display: grid).
- Grid Item: Các phần tử con trực tiếp bên trong container.
- Grid Line: Các đường kẻ (ngang và dọc) tạo nên lưới.
- Grid Track: Khoảng cách giữa 2 đường line liên tiếp (chính là một hàng hoặc một cột).
- Grid Cell: Một "ô" đơn lẻ (giống ô trong Excel).
- Grid Area: Một vùng bao gồm nhiều ô gộp lại.

Grid container:

Mã HTML

```
<body>
  <section>
    <div>Box 1</div>
    <div>Box 2</div>
    <div>Box 3</div>
    <div>Box 4</div>
    <div>Box 5</div>
  </section>
</body>
```

Kết quả hiển thị



Box 1	Box 2	Box 3
Box 4	Box 5	

Mã CSS

```
body{
  background-color: rgba(36, 117, 117, 0.888);
  padding: 10px;
}
section{
  display: inline-grid;
  grid-template-columns: auto auto auto;
  background-color: rgba(203, 137, 46, 0.888);
  padding: 10px;
}
section div{
  border: 1px solid black;
  padding: 5px 10px;
  font-size: 20px;
  background-color: #f0f0f0;
}
```

Grid container:

Mã HTML

```
<body>
  <section>
    <div>Box 1</div>
    <div>Box 2</div>
    <div>Box 3</div>
    <div>Box 4</div>
    <div>Box 5</div>
  </section>
</body>
```

Kết quả hiển thị

Box 1	Box 2	Box 3
Box 4	Box 5	

Mã CSS

```
body{
  background-color: rgba(36, 117, 117, 0.888);
  padding: 10px;
}
section{
  display: grid;
  grid-template-columns: auto auto auto;
  background-color: rgba(203, 137, 46, 0.888);
  padding: 10px;
}
section div{
  border: 1px solid black;
  padding: 5px 10px;
  font-size: 20px;
  background-color: #f0f0f0;
}
```

Grid container:

Mã HTML

```
<section class="container">
  <div class="item">Box 1</div>
  <div class="item">Box 2</div>
  <div class="item">Box 3</div>
  <div class="item">Box 4</div>
  <div class="item">Box 5</div>
</section>
```

Kết quả hiển thị

Box 1	Box 2	Box 3
Box 4	Box 5	

Mã CSS

```
.container{
  background-color: rgba(214, 201, 182, 0.888);
  padding:10px;
  display: grid;
  /* Tạo 3 cột rộng: 200px, tự động, 150px */
  grid-template-columns: 200px auto 150px;
  /* Tạo 2 hàng cao 50px và 100px */
  grid-template-rows: 50px 100px;
}

.item{
  border: 1px solid black;
  padding:5px 10px;
  font-size: 20px;
  background-color: #f0f0f0;
}
```

Đơn vị "fr" (Fractional Unit):

fr đại diện cho một phần của không gian còn trống. Nó thông minh hơn % vì nó tự trừ đi các khoảng cách (gap) trước khi chia.

Mã HTML

```
<section class="container">
  <div class="item">Box 1</div>
  <div class="item">Box 2</div>
  <div class="item">Box 3</div>
  <div class="item">Box 4</div>
  <div class="item">Box 5</div>
</section>
```

Mã CSS

```
.container{
  background-color: rgba(143, 175, 199, 0.888);
  padding: 10px;
  display: grid;
  /* Tạo 3 cột theo tỉ lệ 25% - 50% - 25% */
  grid-template-columns: 1fr 2fr 1fr;
}

.item{
  border: 1px solid black;
  padding: 5px 10px;
  font-size: 20px;
  background-color: #f0f0f0;
}
```

Kết quả hiển thị

Box 1	Box 2	Box 3
Box 4	Box 5	

Hàm repeat():

repeat(3, 1fr) tương đương với 1fr 1fr 1fr.

Mã HTML

```
<section class="container">
  <div class="item">Box 1</div>
  <div class="item">Box 2</div>
  <div class="item">Box 3</div>
  <div class="item">Box 4</div>
  <div class="item">Box 5</div>
</section>
```

Kết quả hiển thị

Box 1	Box 2	Box 3
Box 4	Box 5	

Mã CSS

```
.container{
  background-color: rgba(143, 175, 199, 0.888);
  padding: 10px;
  display: grid;
  grid-template-columns: repeat(3, 1fr);
}
.item{
  border: 1px solid black;
  padding: 5px 10px;
  font-size: 20px;
  background-color: #f0f0f0;
}
```

Hàm minmax():

minmax(100px, auto) nghĩa là: Tối thiểu là 100px, nhưng nếu nội dung dài ra thì cứ tự giãn thoải mái.

Mã HTML

```
<section class="container">
  <div class="item">Box 1</div>
  <div class="item">Box 2</div>
  <div class="item">Box 3</div>
  <div class="item">Box 4</div>
  <div class="item">Box 5</div>
</section>
```

Mã CSS

```
.container{
  background-color: rgba(143, 175, 199, 0.888);
  padding: 10px;
  display: grid;
  grid-template-columns: minmax(80px, 1fr) 2fr 2fr;
}
.item{
  border: 1px solid black;
  padding: 5px 10px;
  font-size: 20px;
  background-color: #f0f0f0;
}
```

Box 1	Box 2	Box 3
Box 4	Box 5	

Khảng cách (Gap):

Thay vì dùng margin cho từng phần tử con, ta dùng gap ở thẻ cha:

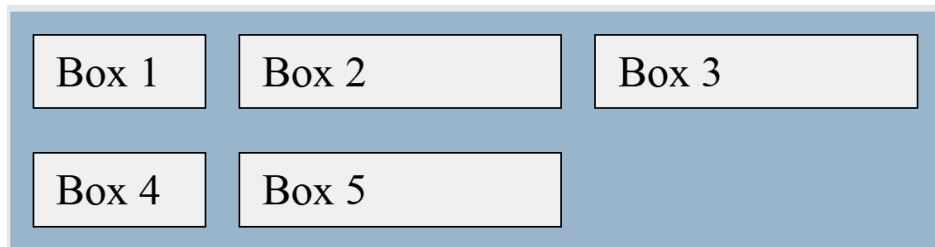
Mã HTML

```
<section class="container">
  <div class="item">Box 1</div>
  <div class="item">Box 2</div>
  <div class="item">Box 3</div>
  <div class="item">Box 4</div>
  <div class="item">Box 5</div>
</section>
```

Mã CSS

```
.container{
  background-color: rgba(143, 175, 199, 0.888);
  padding:10px;
  display: grid;
  grid-template-columns: minmax(80px,1fr) 2fr 2fr;
  column-gap: 20px; /* Khoảng cách cột */
  row-gap: 15px; /* Khoảng cách hàng */
  gap: 20px 15px; /* Viết tắt: [hàng] [cột] */
}
```

Kết quả hiển thị



Định vị phần tử con dựa trên số đường line

Mỗi đường kẻ trong Grid đều được đánh số từ 1.

Mã HTML

```
<section class="container">
  <div class="item it1">Box 1</div>
  <div class="item it2">Box 2</div>
  <div class="item it3">Box 3</div>
  <div class="item it4">Box 4</div>
  <div class="item it5">Box 5</div>
</section>
```

Kết quả hiển thị

Box 1		
Box 2	Box 3	Box 4
Box 5		

Mã CSS

```
.item{
  border: 1px solid black;
  padding: 5px 10px;
  font-size: 20px;
  background-color: #f0f0f0;
}
.it1{
  /* Chiếm từ vạch 1 đến vạch 4 */
  grid-column-start: 1;
  grid-column-end: 4;
  /* Hoặc viết gọn như sau: */
  grid-column: 1 / 4;
  background-color: #ff9999;
}
```

Định vị phần tử con dựa trên grid-template-areas

Mã HTML

```
<body>
  <header>
    <h1>Website của tôi</h1>
  </header>
  <main>Nội dung chính</main>
  <aside>Nội dung phụ</aside>
  <footer>Chân trang</footer>
</body>
```

Mã CSS

```
body{
  background-color: rgba(18, 158, 213, 0.888);
  display: grid;
  grid-template-areas:
    "header header header"
    "main main aside"
    "footer footer footer";
  grid-gap: 10px;
}
body * {
  padding: 10px;
  background-color: #f0f0f0;
}
header { grid-area: header; }
main { grid-area: main; }
aside { grid-area: aside; }
footer { grid-area: footer; }
```

Kết quả hiển thị

Website của tôi	
Nội dung chính	Nội dung phụ
Chân trang	

Căn chỉnh trong Grid

Grid cũng có các thuộc tính căn chỉnh giống Flexbox nhưng mạnh hơn:

- **Căn dọc (Trục vuông góc):** align-items (cho tất cả) hoặc align-self (cho từng thành phần).
- **Căn ngang (Trục chính):** justify-items (cho tất cả) hoặc justify-self (cho từng thành phần).
- **Căn toàn bộ khối lưới:** align-content và justify-content.

Responsive với Grid

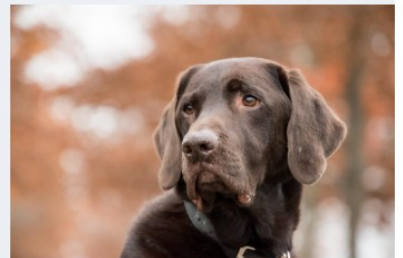
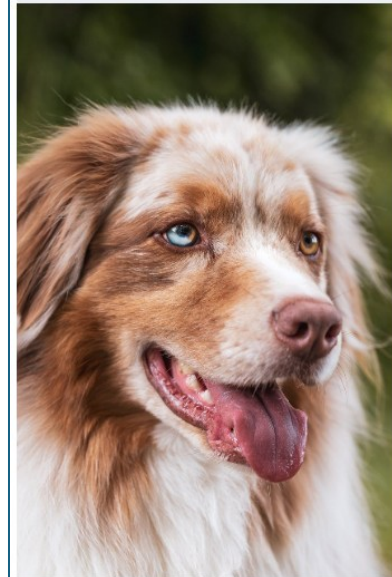
```
.container {  
  display: grid;  
  /* Tự động chia cột sao cho mỗi cột tối thiểu 200px,  
    nếu thiếu chỗ thì tự xuống dòng, nếu thừa chỗ thì tự giãn đều */  
  grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));  
  gap: 20px;  
}
```

Mã HTML

```
<section class="container">
  
  
  
  
</section>
```

Mã CSS

```
.container {
  background-color:rgb(236, 240, 244);
  display: grid;
  grid-template-columns:
    repeat(auto-fill, minmax(200px, 1fr));
  gap: 20px;
  align-content: center;
}
img{ width:100%; height:auto; }
.img-1{ grid-column: span 2; }
.img-3{ grid-row: span 2; }
```



Layout website (bố cục trang web) là khung sườn, cách sắp xếp có chủ đích các thành phần như header, menu, nội dung chính (văn bản, hình ảnh), sidebar và footer trên một trang web, nhằm tạo ra cấu trúc logic, dễ điều hướng và trải nghiệm người dùng hiệu quả.

Các thành phần chính của một trang web:

- Header: Nơi đặt Logo, thanh điều hướng (Menu).
- Navigation (Nav): Thường nằm trong hoặc dưới Header.
- Main Content: Khu vực chứa nội dung chính (Bài viết, sản phẩm).
- Sidebar (Aside): Các thông tin phụ, quảng cáo, hoặc danh mục.
- Footer: Chân trang, chứa thông tin liên hệ và bản quyền.



Các thành phần chính của một trang web:

Để dựng layout, đừng dùng div cho tất cả mọi thứ. Hãy dùng các thẻ "có nghĩa" (Semantic) để Google và các thiết bị hỗ trợ dễ dàng đọc hiểu:

Mã HTML

```
<body>
  <header>Phần Header</header>
  <nav>Phần Menu</nav>
  <main>
    <section>Nội dung 1</section>
    <article>Nội dung 2</article>
  </main>
  <aside>Nội dung phụ</aside>
  <footer>Chân trang</footer>
</body>
```

Hiển thị
sau khi
CSS mẫu



Responsive chính là kỹ thuật giúp trang web của bạn "co giãn" linh hoạt để hiển thị hoàn hảo trên mọi kích thước màn hình: từ đồng hồ thông minh, điện thoại, máy tính bảng đến màn hình TV 4K.



Các bước thực hiện một dự án Responsive

- **Phân tích Layout:** Xác định xem khi thu nhỏ, thành phần nào sẽ biến mất (như menu), thành phần nào sẽ nhảy xuống dưới (như sidebar).
- **Thiết lập Viewport:** Thêm thẻ Meta vào <head>.
- **Code CSS Mobile-First:** Xây dựng giao diện 1 cột, đơn giản nhất.
- **Xác định Breakpoints:** Tìm các điểm mà giao diện bắt đầu trông "xấu" hoặc "khó đọc" để viết Media Queries (Thường là 600px, 768px, 992px, 1200px).
- **Kiểm tra (Testing):** Sử dụng công cụ Inspect Element (F12) trên trình duyệt, chọn biểu tượng điện thoại để test trên nhiều dòng máy khác nhau.

Nền tảng: Thẻ Meta Viewport

Đặt vào trong thẻ <head> của file HTML

```
<head>  
  <meta charset="UTF-8">  
  <meta name="viewport" content="width=device-width, initial-scale=1.0">  
  <title>Responsive</title>  
  <link rel="stylesheet" type="text/css" href="style.css">  
</head>
```

Media Queries

- Media Queries cho phép ta áp dụng các đoạn mã CSS khác nhau cho các kích thước màn hình khác nhau.
- Chúng ta viết CSS cho màn hình nhỏ nhất trước, sau đó dùng Media Queries để thêm các quy tắc cho màn hình lớn hơn.

Ví dụ về Media Queries

Tạo phần menu hiển thị dạng icon trên điện thoại và dạng chữ trên thiết bị khác

Mã HTML

```
<nav>
  <div class="menu-icon">
    
  </div>
  <div class="menu-content">
    <a href="#">Trang chủ</a>
    <a href="#">Giới thiệu</a>
    <a href="#">Dịch vụ</a>
    <a href="#">Liên hệ</a>
  </div>
</nav>
```

Ví dụ về Media Queries

Mã CSS

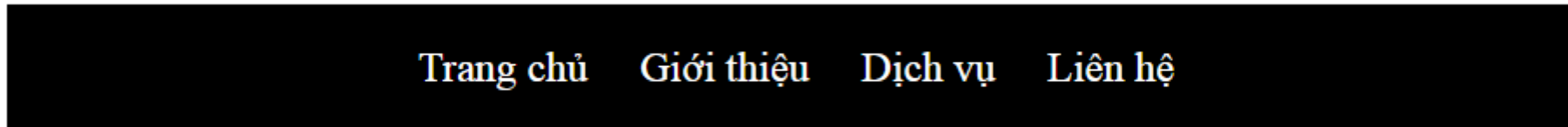
```
/* 1. CSS mặc định (Dành cho điện thoại - dưới 600px) */
nav{background-color: #000000; color: white; padding: 5px 10px;}
.menu-icon { display: block; padding: 5px 10px;}
.menu-icon img { width:40px;}
.menu-content{display: none;}
.menu-content a{color: white; text-decoration: none; padding: 10px;}
/* 2. Tablet (Màn hình từ 600px trở lên) */
@media (min-width: 600px) {
    .menu-icon { display: none; }
    .menu-content {display: flex; justify-content: center; align-items: center;}
}
/* 3. Desktop (Màn hình lớn từ 1024px trở lên) */
@media (min-width: 1024px) {
    nav{ background-color: #0f545a; }
}
```

Ví dụ về Media Queries

Kết quả hiển thị trên màn hình điện thoại



Kết quả hiển thị trên màn hình từ 600px trở lên



Kết quả hiển thị trên màn hình từ 1024px trở lên



Các đơn vị linh hoạt

Đừng bao giờ dùng đơn vị cứng như px cho những thành phần chính. Hãy dùng các đơn vị có khả năng tự co giãn:

- **%**: Tỷ lệ theo phần tử cha.
- **vw** / **vh**: Tỷ lệ theo chiều rộng/chiều cao của toàn bộ cửa sổ trình duyệt (Viewport).
- **em** / **rem**: Tỷ lệ theo kích thước font chữ (Cực kỳ quan trọng để chữ tự to nhỏ theo thiết bị).

Ví dụ:

Thay vì `width: 1200px;`, hãy dùng `max-width: 1200px; width: 100%;`.

Bố cục linh hoạt với Flexbox & Grid

Mã HTML

```
<section class="container">
  <div class="post">
    
    <h2>Tên bài viết 1</h2>
    <p>Phần nội dung tóm tắt cho bài viết số 1.</p>
  </div>
  <div class="post">
    
    <h2>Tên bài viết 2</h2>
    <p>Phần nội dung tóm tắt cho bài viết số 2.</p>
  </div>
  <div class="post">
    
    <h2>Tên bài viết 3</h2>
    <p>Phần nội dung tóm tắt cho bài viết số 3.</p>
  </div>
</section>
```

Mã CSS

```
img {
  width:100%;
  height:auto;
}
.post{
  border: 1px solid #ccc;
  padding: 5px 10px;
  margin-bottom: 10px;
}
.container {
  display: grid;
  gap: 10px;
  grid-template-columns:
    repeat(
      auto-fit,
      minmax(300px, 1fr));
}
```

Kết quả hiển thị trên màn hình mobile



Tên bài viết 1

Phần nội dung tóm tắt cho bài viết số 1.



Tên bài viết 2

Phần nội dung tóm tắt cho bài viết số 2.

Kết quả hiển thị trên màn hình từ 600px trở lên



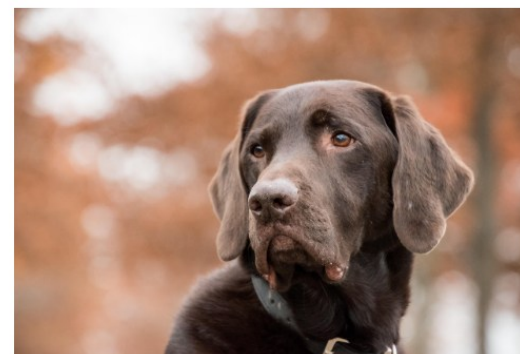
Tên bài viết 1

Phần nội dung tóm tắt cho bài viết số 1.



Tên bài viết 2

Phần nội dung tóm tắt cho bài viết số 2.



Tên bài viết 3

Phần nội dung tóm tắt cho bài viết số 3.

Kết quả hiển thị trên màn hình lớn



Tên bài viết 1

Phần nội dung tóm tắt cho bài viết số 1.



Tên bài viết 2

Phần nội dung tóm tắt cho bài viết số 2.



Tên bài viết 3

Phần nội dung tóm tắt cho bài viết số 3.

Hình ảnh Responsive

Hình ảnh là nguyên nhân chính gây tràn khung. Hãy luôn nhớ quy tắc "vạn năng" này:

Mã CSS

```
img {  
  max-width: 100%;  
  height: auto; /* Giữ đúng tỉ lệ ảnh, không bị méo */  
}
```

Hình ảnh Responsive

Đối với những ảnh cần hiển thị khác nhau trên các thiết bị (ví dụ ảnh ngang cho PC, ảnh dọc cho Mobile), ta nên dùng thẻ <picture> trong HTML:

Mã

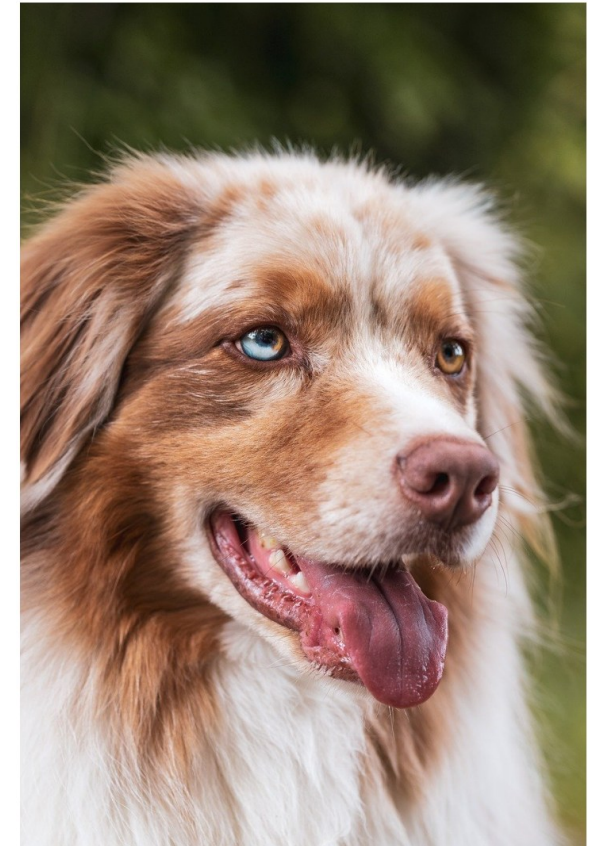
HTML

```
<source media="(min-width: 800px)" srcset="dog1.jpg">  
  
</picture>
```



Hiện trên
màn hình
lớn hơn
800px

Hiện trên
màn hình
Mobile



TÀI LIỆU THAM KHẢO

1. CSS: <https://www.w3schools.com/css/>
2. CSS tutorial: https://hocwebchuan.com/tutorial/tutorial_css.php
3. CSS3: <https://hocwebchuan.com/tutorial/css3/>
4. SASS: <https://www.w3schools.com/sass/>
5. SCSS tutorial: <https://hocwebchuan.com/tutorial/scss/>

THỰC HÀNH TỔNG HỢP

Thực hiện UI cho các trang trong dự án web của nhóm.

Các trang gợi ý:

- Trang đăng nhập
- Trang chính
- Trang chi tiết bài viết/sản phẩm
- Trang quản trị
- Trang quản lý bài viết/sản phẩm
- Trang quản lý tài khoản
- Trang thông tin tài khoản
-



THANKS
